



Input Mode System & Global Controls

A full account of every file created, rewritten, and extended in the Alpha 0.2 input session.

SESSION	Alpha 0.2 — Input Mode System
DATE	May 11, 2026
STATUS	Running — no errors
SCOPE	10 files changed (4 new, 5 rewritten, 1 extended)

Contents

- 1. Core Type System Expansion
- 2. Input Mode Context
- 3. Gesture Mapper
- 4. Gesture Engine
- 5. Global Command Box
- 6. Input Mode Indicator
- 7. Sidebar Layout
- 8. Settings Page
- 9. Command Router
- 10. App Root
- 11. Gesture '!' Command Reference Table
- 12. File Manifest
- 13. What Remains Mocked

1. Core Type System Expansion · `src/core/types.ts`

Extended the shared type contract. All runtime layers import from this single source of truth.

Added types:

<code>InputMode</code>	"voice" "gesture" "text" "hybrid" — the four OS-level input profiles
<code>InputModeConfig</code>	Carries voiceEnabled, gestureEnabled, textEnabled, commandBoxVisible booleans
<code>GestureMapping</code>	Typed binding of GestureType !' command string !' InputSource
<code>SWIPE_UP / SWIPE_DOWN / SWIPE_ACROSS navigation_action</code>	Three new GestureType enum values
<code>scroll_action</code>	New CommandIntent — gesture-driven panel navigation
<code>"scroll"</code>	New CommandIntent — gesture-driven scroll commands
	New action value in CommandResult alongside navigate, toggle, launch

2. Input Mode Context · `src/core/input-mode.tsx` [NEW FILE]

A React context that manages all input mode state across the OS. Wraps the entire app via InputModeProvider in App.tsx.

State managed (all persisted to localStorage):

- mode — selected profile (hybrid / voice / gesture / text)
- voiceActive — independent microphone toggle, separate from mode profile
- gestureActive — independent camera/gesture toggle, separate from mode profile
- isCommandBoxOpen — whether the floating command box is visible
- keyboardOnly — derived: true when both voice and gesture are off

Key design decision:

voiceActive and gestureActive are independent of the mode profile. A user can stay on Hybrid mode but flip the microphone off. The mode label reflects preferred style; the hardware toggles reflect what is actually running. Both are stored in separate localStorage keys so they survive mode switches.

Exported API:

- `setMode(mode)` — switch mode profile
- `cycleMode()` — rotate through all four modes in order
- `toggleVoice()` — flip microphone state
- `setVoice(bool)` — set microphone state explicitly
- `toggleGesture()` — flip camera state
- `setGesture(bool)` — set camera state explicitly
- `toggleCommandBox()` — open or close the floating command box
- `openCommandBox()` — open explicitly
- `closeCommandBox()` — close explicitly

Global keyboard shortcut:

Ctrl+Space toggles the command box from anywhere in the OS. Escape closes it. Both registered via a single useEffect in the provider — no per-page listener setup needed.

INPUT_MODE_META registry:

Each mode has a label, shortLabel, description, and Tailwind color/border/bg tokens. Any component can render mode-appropriate styling without hardcoding colors — they import from this registry.

3. Gesture Mapper · [src/lib/gesture-mapper.ts](#) [NEW FILE]

The canonical layer converting raw `GestureType` values into natural language command strings fed into `zaraRuntime.executeCommand()`. This is the clean interface between hardware recognition and the command pipeline — when real `MediaPipe` integration arrives, nothing in this file changes.

PANEL_ORDER array:

Defines the canonical left-to-right order of all 10 ZaraOS panels. Used by `getPreviousPanel()` and `getNextPanel()` helpers so `SWIPE_LEFT/RIGHT` always resolve to the correct target regardless of which panel is currently active.

Gesture !' Command Reference:

Gesture	Command fed to Zara Runtime	Notes
OPEN_PALM	open assistant	Always opens /assistant
SWIPE_LEFT	navigate to /[previous panel]	Panel-aware via <code>getPreviousPanel()</code>
SWIPE_RIGHT	navigate to /[next panel]	Panel-aware via <code>getNextPanel()</code>
SWIPE_UP	scroll down	Content moves upward
SWIPE_DOWN	scroll up	Content moves downward
SWIPE_ACROSS	go home	Closes active window
PINCH	select focused	Selects focused element
GRAB	begin drag	Requires gesture mode
FIST	go home	Dismiss / go home
TWO_FINGERS_UP	enable precision scroll	Requires gesture mode

GESTURE_MAPPINGS export:

Full typed mapping table consumed by the Settings app. Includes `gesture`, `label`, `description`, `command`, `source`.

4. Gesture Engine · [src/lib/gesture-engine.ts](#) [REWRITTEN]

Upgraded from a thin placeholder to a properly structured, integration-ready class. All gesture events dispatch through the same runtime pipeline as voice and keyboard.

New capabilities:

- `setCurrentPath(path)` — engine knows the current panel for accurate swipe navigation
- `onGesture(callback)` — callback now receives both `GestureType` and the resolved command string
- `onStatusChange(callback)` — separate callback for tracking start/stop state changes
- 600ms debounce window per gesture — prevents rapid double-fires from hand jitter
- `simulateGestureSequence(gestures[], delayMs)` — fires a series with configurable delay
- `isActive()` / `getLastGesture()` / `getLastGestureLabel()` — readable state for UI displays

MediaPipe integration point (clearly marked in source):

1. Import `@mediapipe/hands`
2. Create `Hands` instance with model complexity and confidence thresholds
3. Feed webcam frames via `requestAnimationFrame` loop
-

- 4. Classify hand landmarks !' GestureType
- 5. Call this.dispatchGesture(classified) — no other changes needed

5. Global Command Box · `src/components/global-command-box.tsx` [NEW FILE]

A persistent floating text input accessible from every panel. The privacy-first alternative to voice — lets users type natural language commands without making any sound or activating the microphone.

- Triggered by Ctrl+Space keyboard shortcut or the Command Box sidebar button
- Slides up from the bottom with glass/blur backdrop overlay
- Full-width panel constrained to max-w-2xl, centred on screen
- Header bar with ZARA COMMAND label and current input mode badge
- Large clean input field with auto-focus on open
- Arrow Up/Down navigates command history — last 5 commands stored
- Three most recent commands shown as clickable history entries above input
- Six quick-fill suggestion chips for the most common commands
- Mic icon button (voice input integration point, to be wired in Alpha 0.4)
- Status line showing Zara's live status and a privacy confirmation note

All submitted commands call `zaraRuntime.executeCommand(text, "keyboard")` — same pipeline as voice and gesture. If the result has action: "navigate", the box closes and navigates automatically. No separate command system.

6. Input Mode Indicator · [src/components/input-mode-indicator.tsx](#) [NEW FILE]

A compact sidebar widget showing the current input mode with an inline dropdown picker.

- Displays current mode icon (Layers/Mic/Hand/Keyboard), short label, three mini input-channel badges
- Clicking opens a dropdown listing all four modes with active dot indicators
- Double-clicking cycles modes via `cycleMode()`
- Fully mode-color-aware — reads `INPUT_MODE_META` tokens, no hardcoded colors

7. Sidebar Layout · [src/components/layout.tsx](#) [REWRITTEN]

The bottom section of the sidebar was restructured to surface input controls at the OS level, always visible from every panel.

New Input Hardware section:

- Section label `INPUT HARDWARE` in muted monospace
- Voice toggle — amber when active (Mic icon + glowing amber dot), muted with `MicOff` icon when disabled
- Gesture toggle — purple when active (Hand icon + glowing purple dot), muted when disabled
- Both buttons are side-by-side, half-width, with immediate visual state feedback
- Keyboard only green badge auto-appears when both are off — one glance confirms privacy state
- All toggle state persists immediately to `localStorage` via the `InputMode` context

Wiring added to Layout:

- `gestureEngine.setCurrentPath(location)` — synced on every route change via `useEffect`
- `gestureEngine.onGesture()` !' `zaraRuntime.executeCommand()` — gesture/runtime bridge, registered once
- `useInputMode()` provides `voiceActive`, `gestureActive`, `keyboardOnly`, `toggleVoice`, `toggleGesture`

Sidebar render order (bottom section, top to bottom):

1. Divider rule
2. Input Hardware — Voice and Gesture toggle buttons + Keyboard-only badge
3. Input Mode Indicator — mode profile selector dropdown
4. Command Box button — `Ctrl+Space` shortcut label shown
5. System Status — green pulse dot

8. Settings Page · [src/pages/settings.tsx](#) [REWRITTEN]

Added a full Input Mode tab and rewrote the Gestures tab. The Settings page now mirrors all sidebar controls in a more detailed form.

New Input Mode tab:

- Mode selector — 4 cards in a 2×2 grid (Hybrid, Voice, Gesture, Text)
- Each card shows the mode icon, label, description, and a checkmark when active
- Hardware Input card — Voice, Gesture, and Keyboard rows with toggle Switches
- Voice row — live description text changes based on state (active vs disabled)
- Gesture row — same treatment for camera
- Keyboard row — permanently shown as Always On with a locked switch
- Keyboard-only mode active notice appears in green when both hardware inputs are off
- Command Box shortcut card — displays `Ctrl + Space` key combination
- Architecture note — explains all modes share the same Runtime pipeline

Gestures tab (fully rewritten):

- Camera tracking enable/disable switch wired to `gestureEngine.startTracking()` / `stopTracking()`
-

Live indicator showing the most recently tested gesture name

- Full gesture map table — all 9 gestures with label, description, resolved command, Test button
- Test buttons call `gestureEngine.simulateGesture()` and update the live indicator
- Gesture Mode badge on gestures requiring gesture mode active
- MediaPipe integration note pointing to the exact file and function name

9. Command Router · `src/lib/command-router.ts` [REWRITTEN]

Expanded the intent rule set to handle all gesture-generated commands, plus 30+ total natural language intent rules across all panels.

New intent rules added:

- Navigation rules — one exact-match rule per panel path at 99% confidence
- go home / dismiss / close active window !' navigation_action !' target: /
- scroll down / scroll up / enable precision scroll !' scroll_action intent
- select focused / begin drag !' navigation_action (gesture meta-commands)
- open assistant / wake zara !' open_app !' target: /assistant (OPEN_PALM gesture)
- Heuristic: question-phrased input !' ai_question intent at 85% confidence

getResponseText() function:

Maps every CommandIntent to a readable response string. navigation_action and scroll_action have their own response generators. This keeps response logic out of the runtime.

10. App Root · `src/App.tsx` [UPDATED]

InputModeProvider added to the provider stack. Positioned inside RuntimeProvider (components using useInputMode may also call useRuntime) and outside PrivacyProvider.

```
QueryClientProvider
  RuntimeProvider
    InputModeProvider      !• new
    PrivacyProvider
    TooltipProvider
    Router
```

11. File Manifest

STATUS	FILE
New	<code>src/core/input-mode.tsx</code>
New	<code>src/lib/gesture-mapper.ts</code>
New	<code>src/components/global-command-box.tsx</code>
New	<code>src/components/input-mode-indicator.tsx</code>
Rewritten	<code>src/lib/gesture-engine.ts</code>
Rewritten	<code>src/components/layout.tsx</code>
Rewritten	<code>src/lib/command-router.ts</code>
Rewritten	<code>src/pages/settings.tsx</code>
Rewritten	<code>src/App.tsx</code>
Extended	<code>src/core/types.ts</code>

12. What Remains Mocked

The following are architecture placeholders — integration points are clearly commented in source.

Voice

VoiceEngine still uses a stub. The toggle changes UI state but does not yet call `getUserMedia()`.

Gesture

GestureEngine uses `simulateGesture()`. All UI and routing works via simulation. No webcam access yet.

AI

All Zara responses are mocked. AIEngine has integration points but calls no external APIs.

Backend

No backend calls. All state lives in `localStorage`. `api-server` artifact not yet used by ZaraOS.

